

Receipt & Invoice Digitizer (MydigiBill)

Milestone- 2 Documentation

1. Introduction to Milestone-2

Milestone-2 represents the **core intelligence phase** of the MydigiBill project. While Milestone-1 focused on converting physical receipts into digital text using OCR, Milestone-2 focuses on **understanding** that text.

Raw OCR output is inherently **unstructured**. It contains words, numbers, symbols, and line breaks without meaning. Milestone-2 introduces **semantic understanding**, enabling the system to identify:

- What the text represents
- Which numbers are prices
- Which lines are purchased items
- Whether the receipt totals are mathematically valid

This milestone is critical because **all future analytics, insights, and reporting depend on the correctness of extracted data.**

2. Purpose and Importance of Milestone-2

2.1 Why OCR Alone Is Not Enough

OCR only answers:

“What characters are present in the image?”

But a financial application must answer:

- Which characters represent money?
- Which numbers are totals vs item prices?
- Which words represent business entities?
- Is the receipt logically correct?

Milestone-2 bridges this gap by converting **text** → **information** → **validated data**.

3. Milestone-2 Functional Requirements

Milestone-2 satisfies the following requirements:

- Apply **Regular Expressions (Regex)** for deterministic pattern extraction
 - Apply **Natural Language Processing (NLP)** for semantic understanding
 - Extract **key receipt fields**
 - Detect **line items and prices**
 - Validate **subtotal, tax, and total relationships**
 - Detect **duplicate receipts**
 - Store **structured results in a relational database**
-

4. Architectural Overview of Milestone-2

Milestone-2 operates as a **processing pipeline**, where each stage improves data quality.

Processing Stages:

1. OCR Text Input
2. Text Cleaning & Normalization
3. Regex-Based Extraction
4. NLP-Based Entity Recognition
5. Hybrid Field Mapping
6. Validation Engine
7. Duplicate Detection
8. Database Persistence

Each stage is **independent yet sequential**, making the system modular and extensible.

5. Text Cleaning & Normalization (Pre-Processing Stage)

5.1 Nature of OCR Noise

OCR output commonly contains:

- Broken words (Pupusa Q ueso)
- Extra spaces
- Inconsistent capitalization
- Misrecognized characters (O instead of 0)
- Random line breaks

If not handled, these issues severely degrade extraction accuracy.

5.2 Cleaning Techniques Applied

- Removal of empty lines
- Trimming whitespace
- Normalizing spacing
- Consistent text casing
- Line-by-line segmentation

This step ensures downstream components receive **clean and predictable input**.

6. Regular Expression (Regex) Parsing

6.1 Why Regex Is Used

Regex provides **rule-based certainty**. Financial data follows predictable patterns:

- Prices have decimals
- Totals follow keywords
- Items appear on separate lines

Regex ensures **speed, accuracy, and offline reliability**.

6.2 Line-Item Extraction Using Regex

Each receipt line is scanned to match patterns like:

Item Name 12.99

Rules Applied:

- Line must end with a numeric price
- Exclude lines containing:
 - Total
 - Subtotal
 - Tax
 - Change
- Remove leading quantities (2 Burger)
- Deduplicate item-price pairs

This logic ensures only **genuine purchasable items** are extracted.

6.3 Subtotal Extraction

Regex searches for keywords like:

- Subtotal
- Sub Total

The first valid monetary value following the keyword is extracted.

6.4 Tax Extraction

Tax values may appear:

- Once
- Multiple times (state + local tax)

The system:

- Extracts all tax values
 - Aggregates them into a single tax amount
-

6.5 Total Extraction

The total amount is identified using keywords:

- Total
- Grand Total

If multiple numeric values exist, the **last valid occurrence** is selected to avoid subtotal confusion.

7. NLP Integration Using spaCy

7.1 Why NLP Is Necessary

Regex cannot reliably detect:

- Business names
- Dates in multiple formats
- Time expressions
- Semantic entities

NLP solves this by understanding **language context**, not just patterns.

7.2 spaCy Model Selection

The project uses:

en_core_web_sm

Reasons:

- Lightweight
- Fast execution
- Pretrained Named Entity Recognition (NER)
- Ideal for local systems

7.3 Entities Extracted Using NLP

Entity Purpose

ORG Detect store/merchant name

DATE Detect receipt date

TIME Detect transaction time

spaCy analyzes the entire OCR text and identifies entity spans automatically.

7.4 Date Normalization

Extracted dates may appear as:

- 09-May-2017
- 09/05/2017
- May 9, 2017

The system attempts multiple formats and converts them into a **standardized format** for database consistency.

8. Hybrid Extraction Strategy

8.1 Why Hybrid Approach Is Required

No single technique is perfect:

Method	Strength	Limitation
Regex	Accurate for numbers	Cannot understand meaning
NLP	Understands context	Less precise for amounts

8.2 Hybrid Strategy

- Regex → Prices & totals
- NLP → Store, date, time
- Rules → Payment mode

This results in **maximum accuracy with minimal overhead**.

9. Payment Mode Detection

Payment type is inferred using keyword matching:

- Cash
- Card
- Visa
- MasterCard
- UPI

This rule-based approach avoids unnecessary AI dependency.

10. Line-Item Integrity & Deduplication

10.1 Deduplication Need

OCR may repeat lines due to scanning artifacts.

10.2 Solution

- Each item is stored using a unique (name + price) key
- Duplicate entries are ignored

This ensures **clean item lists**.

11. Total Validation Engine

11.1 Validation Formula

Subtotal + Tax $\approx$ Total

11.2 Tolerance Handling

A tolerance of ± 0.05 is used to account for:

- Rounding differences
 - OCR inaccuracies
-

11.3 Validation Outcomes

Condition	Action
-----------	--------

Valid totals	Allow save
--------------	------------

Invalid totals	Display warning, disable save
----------------	-------------------------------

Users can still **review extracted data** even if totals are invalid.

12. Required Field Validation

Fields validated:

- Store
- Subtotal
- Total
- Items

This ensures **minimum data completeness**.

13. Duplicate Receipt Detection

13.1 Detection Logic

- Receipt filename checked against database
- Prevents duplicate storage

This protects:

- Analytics accuracy
- Database integrity

14. Validation Feedback System

Validation results are displayed as **visual banners**, showing:

- Status (Pass/Fail)
- Clear explanation

This transparency improves:

- User trust
 - Debugging clarity
 - System usability
-

15. Structured Data Model

15.1 Receipt Table

Stores:

- Receipt metadata
- Financial totals
- Payment information
- Timestamp

15.2 Receipt Items Table

Stores:

- Item name
- Item price
- Foreign key to receipt

This relational design supports **future analytics and reporting**.

16. Database Storage Workflow

1. Validate receipt
2. Insert receipt record
3. Retrieve receipt ID
4. Insert item records
5. Commit transaction

Atomic operations ensure **data consistency**.

17. Error Handling & Fallback Mechanisms

- If AI extraction fails → Rule-based parser used
- If NLP fails → Regex still extracts prices
- If totals invalid → Saving disabled, UI visible

This makes the system **fault-tolerant**.

18. Observations & Results

- High accuracy across restaurant receipts
 - Robust against OCR noise
 - Consistent database records
 - Fully offline capable
-

19. Milestone-2 Achievements

- ✓ Regex + NLP extraction
 - ✓ Line-item detection
 - ✓ Validation engine
 - ✓ Duplicate prevention
 - ✓ Structured storage
 - ✓ User-friendly feedback
-

20. Conclusion

Milestone-2 transforms MydigiBill into an **intelligent financial document processor**. By combining deterministic rules with linguistic understanding, the system achieves **accuracy, reliability, and scalability**, forming the backbone for analytics and insights in future milestones